

Lista de Revisão — Programação II
Prof. Rodrigo P. S. Ribeiro

Orientações gerais

- Os exercícios podem ser desenvolvidos na linguagem de programação escolhida pelo aluno.
- Quando necessário, podem ser utilizados frameworks e bibliotecas, desde que o aluno saiba explicar seu funcionamento.
- Em exercícios com banco de dados, pode ser utilizado MySQL, PostgreSQL, SQLite ou outro SGBD relacional.
- Nos exercícios web, utilize HTML e CSS e, quando necessário, uma linguagem de programação no cliente ou servidor.
- Organize o código utilizando nomes significativos, indentação adequada e separação de responsabilidades.
- Sempre que possível, valide os dados fornecidos pelo usuário.

1. Função para análise de números

Crie um programa que receba uma lista de números inteiros e utilize funções para determinar:

- a) o maior valor;
- b) o menor valor;
- c) a média dos valores;
- d) a quantidade de números pares;
- e) a quantidade de números ímpares.

Cada operação deve ser implementada em uma função diferente.

2. Validação utilizando funções

Desenvolva um programa de cadastro de usuário que solicite:

- a) nome;
- b) idade;
- c) e-mail;
- d) senha.

Implemente funções independentes para validar cada informação.

Regras mínimas:

- o nome não pode estar vazio;
- a idade deve estar entre 14 e 120 anos;
- o e-mail deve possuir um formato minimamente válido;
- a senha deve possuir pelo menos 8 caracteres.

O programa somente deverá concluir o cadastro quando todos os dados forem válidos.

3. Calculadora modular

Construa uma calculadora utilizando funções para realizar as operações:

- soma;
- subtração;
- multiplicação;
- divisão;
- potência;
- resto da divisão.

O programa deverá apresentar um menu e permitir que o usuário execute várias operações até escolher a opção de encerrar.

Trate situações inválidas, como divisão por zero.

4. Fatorial recursivo

Implemente uma função recursiva que calcule o fatorial de um número inteiro não negativo.

Exemplo:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Além do programa, responda:

1. Qual é o caso-base da função?
2. O que aconteceria se o caso-base não existisse?
3. Qual seria uma versão não recursiva do mesmo algoritmo?

5. Sequência de Fibonacci

Crie uma função recursiva que calcule o n-ésimo termo da sequência de Fibonacci.

Depois, implemente uma segunda versão utilizando repetição.

Compare as duas soluções e explique qual tende a ser mais eficiente para valores elevados de n.

6. Soma recursiva de uma lista

Implemente uma função recursiva que receba uma lista de números e retorne a soma de todos os seus elementos.

Exemplo:

Entrada:

[10, 20, 5, 3]

Saída:

38

Não utilize estruturas de repetição dentro da função recursiva.

7. Gerenciador de lista de tarefas

Crie um programa que utilize uma estrutura de dados do tipo lista para armazenar tarefas.

O programa deverá permitir:

- adicionar tarefa;
- listar tarefas;
- buscar uma tarefa;
- alterar uma tarefa;
- remover uma tarefa;
- informar a quantidade total de tarefas.

Desafio adicional: permita marcar uma tarefa como concluída.

8. Histórico utilizando pilha

Implemente uma estrutura de pilha para representar o histórico de páginas visitadas por um usuário.

O sistema deverá permitir:

- visitar uma nova página;
- visualizar a página atual;
- voltar para a página anterior;
- exibir o histórico.

Explique por que uma pilha é adequada para esse problema.

9. Sistema de atendimento utilizando fila

Implemente um sistema de atendimento utilizando uma fila.

Cada pessoa deverá possuir:

- número da senha;
- nome;
- horário de entrada.

O sistema deverá permitir:

- adicionar uma pessoa à fila;
- chamar a próxima pessoa;
- consultar quem será o próximo;
- mostrar toda a fila;
- informar quantas pessoas estão aguardando.

Explique por que uma fila é mais adequada que uma pilha neste problema.

10. Comparando lista, pilha e fila

Considere os seguintes problemas:

- A. Histórico do navegador.
- B. Pessoas aguardando atendimento.
- C. Catálogo de produtos.
- D. Função “desfazer” de um editor de texto.
- E. Playlist de músicas que pode ser acessada por posição.

Para cada situação, indique qual estrutura seria mais adequada entre:

- lista;
- pilha;
- fila.

Justifique cada resposta.

11. Orientada a objetos

Crie uma classe Produto contendo os atributos:

- i. código;
- ii. nome;
- iii. preço;
- iv. quantidade em estoque.

Implemente métodos para:

- adicionar unidades ao estoque;
- retirar unidades do estoque;
- alterar o preço;

- calcular o valor total armazenado daquele produto.

Não permita retirar uma quantidade superior ao estoque disponível.

Crie pelo menos três objetos para testar a classe.

12. Sistema orientado a objetos

Modele um pequeno sistema escolar utilizando orientação a objetos.

Crie pelo menos as seguintes classes:

Pessoa

- ❖ nome;
- ❖ data de nascimento.

Aluno

- ❖ matrícula;
- ❖ curso;
- ❖ notas.

Professor

- ❖ matrícula;
- ❖ disciplina.

Utilize herança para representar a relação entre Pessoa, Aluno e Professor.

Crie métodos apropriados, como cálculo da média do aluno e apresentação dos dados de cada objeto.

13. Modelagem de banco de dados

Projete um banco de dados para uma biblioteca.

O sistema deverá armazenar informações sobre:

- a) livros;
- b) autores;
- c) usuários;
- d) empréstimos.

Defina:

- tabelas;
- campos;
- tipos de dados;
- chaves primárias;

- chaves estrangeiras;
- relacionamentos.

Em seguida, escreva os comandos SQL necessários para criar as tabelas utilizando CREATE TABLE.

14. INSERT e SELECT

Utilizando o banco de dados da biblioteca do exercício anterior:

1. Cadastre pelo menos 5 livros.
2. Cadastre pelo menos 3 usuários.
3. Cadastre pelo menos 3 empréstimos.

Utilize comandos INSERT.

Depois escreva consultas SELECT para:

- listar todos os livros;
- mostrar somente título e autor;
- buscar livros de determinado autor;
- mostrar os livros ordenados alfabeticamente;
- mostrar apenas livros disponíveis.

15. UPDATE e DELETE

Utilizando o mesmo banco de dados:

Crie comandos SQL para:

- alterar o nome de um usuário;
- alterar a disponibilidade de um livro;
- atualizar a data de devolução de um empréstimo;
- excluir um usuário;
- excluir um livro.

Antes de executar os comandos DELETE, analise possíveis problemas relacionados às chaves estrangeiras.

Explique o que pode acontecer ao tentar excluir um registro relacionado a outros registros.

16. Consultas relacionando tabelas

Utilizando o banco da biblioteca, crie consultas SQL que apresentem:

1. nome do usuário e título do livro emprestado;
2. todos os empréstimos realizados;

3. empréstimos ainda não devolvidos;
4. quantidade de empréstimos realizada por usuário;
5. livros que nunca foram emprestados.

Utilize JOIN sempre que necessário.

17. Página HTML de apresentação

Crie uma página HTML para apresentar um curso técnico.

A página deverá possuir obrigatoriamente:

- cabeçalho;
- menu;
- título principal;
- subtítulos;
- parágrafos;
- imagem;
- lista;
- tabela;
- formulário;
- rodapé.

Utilize tags HTML semânticas sempre que possível, como:

header, nav, main, section, article e footer.

18. Estilização com CSS

Utilizando a página criada no exercício anterior, desenvolva um arquivo CSS separado.

A página deverá possuir:

- definição de fontes;
- cores;
- espaçamentos;
- bordas;
- estilização do menu;
- estilização de botões;
- efeito hover;
- uso de class;

- uso de id;
- layout utilizando Flexbox ou Grid;
- adaptação básica para telas menores.

Não utilize estilos diretamente nas tags HTML.

19. Requisição HTTP e API

Escolha uma API pública e desenvolva um programa que faça uma requisição HTTP utilizando o método GET.

Exemplos de dados que podem ser consultados:

- previsão do tempo;
- CEP;
- países;
- moedas;
- personagens;
- filmes;
- livros.

O programa deverá:

1. realizar a requisição;
2. verificar o código de resposta HTTP;
3. interpretar os dados recebidos;
4. apresentar algumas informações ao usuário;
5. tratar possíveis erros.

Depois responda:

- Qual URL foi utilizada?
- Qual método HTTP foi utilizado?
- Qual código HTTP representa uma requisição bem-sucedida?
- Em qual formato os dados foram recebidos?
- Qual é a diferença entre uma requisição e uma resposta HTTP?

20. Finalizando - Sistema Web Cliente-Servidor

Desenvolva uma pequena aplicação web que reúna os principais conhecimentos da disciplina.

Escolha um dos temas:

- sistema de biblioteca;
- controle de estoque;
- agenda de contatos;
- sistema de tarefas;
- cadastro de alunos;
- sistema de reservas;
- catálogo de jogos;
- outro tema aprovado pelo professor.

Requisitos obrigatórios

O sistema deverá possuir uma arquitetura cliente-servidor.

Cliente

Desenvolva uma interface utilizando:

- HTML;
- CSS;
- formulários;
- tabelas ou listas para apresentação de informações.

Servidor

Implemente funcionalidades para:

- receber requisições HTTP;
- processar os dados;
- validar as informações;
- acessar o banco de dados;
- devolver respostas ao cliente.

Banco de dados

O sistema deverá permitir pelo menos as quatro operações básicas de CRUD:

- CREATE — cadastrar um novo registro;
- READ — consultar registros;
- UPDATE — alterar um registro;
- DELETE — excluir um registro.

Orientação a objetos

Utilize pelo menos uma classe para representar alguma entidade do sistema.

Exemplo:

Produto, Aluno, Livro, Usuário ou Tarefa.

Estruturas de dados

Utilize pelo menos uma estrutura entre:

- lista;
- pilha;
- fila.

A utilização deverá fazer sentido dentro do problema escolhido.

Requisições HTTP

A aplicação deverá utilizar pelo menos:

- uma requisição GET;
- uma requisição POST;
- uma operação de alteração;
- uma operação de exclusão.

Podem ser utilizados PUT, PATCH e DELETE, conforme a tecnologia escolhida.

Entrega

O aluno deverá apresentar:

1. código-fonte;
2. banco de dados;
3. interface funcionando;
4. explicação da arquitetura cliente-servidor utilizada;
5. principais rotas/endpoints do sistema;
6. demonstração das operações CRUD;
7. explicação de pelo menos uma classe criada;
8. explicação de uma estrutura de dados utilizada.

Questões para revisão conceitual

Durante a resolução dos exercícios, o aluno deverá conseguir explicar os seguintes conceitos:

- diferença entre função e procedimento;
- parâmetros e valores de retorno;
- caso-base em uma função recursiva;

- diferença entre recursão e repetição;
- funcionamento de listas, pilhas e filas;
- conceitos de LIFO e FIFO;
- classes, objetos, atributos e métodos;
- encapsulamento e herança;
- banco de dados relacional;
- chave primária e chave estrangeira;
- comandos CREATE, INSERT, SELECT, UPDATE e DELETE;
- relacionamento entre tabelas e JOIN;
- diferença entre cliente e servidor;
- funcionamento básico do protocolo HTTP;
- métodos GET, POST, PUT/PATCH e DELETE;
- códigos de status HTTP;
- HTML semântico;
- seletores CSS;
- Flexbox e/ou Grid;
- funcionamento geral de uma aplicação web integrada a banco de dados.